Architectural Paradigms for High-Frequency Algorithmic Trading Systems Utilizing the Upstox V3 API Ecosystem

1. Introduction: The Evolution of Algorithmic Infrastructure

The landscape of retail algorithmic trading in India has undergone a seismic shift, driven by the democratization of access to low-latency exchange connectivity. The transition from legacy, terminal-based trading to API-driven architectures has empowered quantitative traders to deploy sophisticated strategies that were previously the domain of high-frequency trading (HFT) firms. Upstox, a vanguard in this technological revolution, has released its V3 API ecosystem, a platform designed to handle the rigorous demands of modern quantitative finance. This report provides an exhaustive technical analysis of the Upstox API V3, focusing on the architectural requirements for building a robust, automated trading system. It synthesizes advanced quantitative models—specifically Neuro-Quantum logic, Kalman filtering, and Fractal analysis—with the practical engineering challenges of authentication, data ingestion, and order execution.

The analysis draws upon a comprehensive review of technical documentation, proprietary strategy papers (The Neuro-Quantum Brain, The Rocket Science Trend Kalman Filter), and community developer discussions. It addresses the critical engineering challenges of handling binary WebSocket streams via Protocol Buffers (Protobuf), managing OAuth 2.0 security lifecycles, and constructing reactive user interfaces using Streamlit.

1.1 The Shift from JSON to Protobuf

A defining characteristic of the Upstox V3 architecture is the adoption of Google's Protocol Buffers (Protobuf) for real-time market data delivery.[1] In traditional V2 REST architectures, market data was transmitted as verbose JSON text. While human-readable, JSON incurs significant bandwidth overhead and parsing latency, particularly during periods of high market volatility when tick density explodes.

The V3 implementation utilizes binary serialization. Data is encoded into compact binary packets based on a pre-defined schema (.proto file). This shift necessitates a fundamental redesign of the client-side ingestion engine. The receiving application must now possess a compilation layer to decode these binary streams into usable objects. This report analyzes the implications of this shift, demonstrating that while it increases initial development complexity,

it reduces payload size by orders of magnitude, thereby lowering the "tick-to-trade" latency—a critical metric for the "Sniper" strategies discussed in the research material.[3]

---

2. Authentication and Security Architecture

The foundation of any algorithmic trading system is a secure, resilient authentication mechanism. Upstox utilizes the OAuth 2.0 protocol, the industry standard for delegated authorization. This protocol ensures that the trading algorithm can operate with the user's permissions (scopes) without strictly retaining the user's raw login credentials, although the automation of this process introduces specific engineering challenges regarding Two-Factor Authentication (2FA).

2.1 The OAuth 2.0 Authorization Code Flow

The authentication lifecycle in Upstox is strictly defined by the Authorization Code Grant type. This flow is designed to separate the role of the resource owner (the trader) from the client (the trading bot).[4]

2.1.1 The Handshake Mechanism

The process initiates with the construction of an authorization URL. This URL must explicitly include the client_id (API Key), the redirect_uri (which acts as a security whitelist), and the response_type set to code.[4]

The robust nature of this flow lies in the redirect_uri. If a malicious actor intercepts the request, they cannot receive the authorization code unless they also control the server at the redirected address.

Parameter	Description	Security Implication
client_id	Public identifier for the app	Exposed in the URL; not a secret.
redirect_uri	Callback URL	Must match the dashboard config exactly to prevent open redirect attacks.
state	Random string	Prevents Cross-Site Request Forgery (CSRF) by maintaining state between request and callback.

scope	Permissions requested	Limits the potential damage if the token is compromised (Principle of Least Privilege).

Upon successful user login, Upstox redirects the browser to the callback URL with a temporary code. This code is short-lived and must be exchanged immediately for an access_token via a server-to-server POST request. This separation ensures that the access_token—the actual key to the account—is never exposed in the browser URL bar.[5]

2.2 Automating the "Headless" Login

For algorithmic trading, the requirement for manual user intervention (entering a password and OTP) daily is a significant friction point. While regulatory frameworks (SEBI) emphasize user supervision, technical implementations often seek to automate this "headless" login process using Time-based One-Time Passwords (TOTP).[6]

2.2.1 TOTP Algorithmic Implementation

The Time-based One-Time Password algorithm (RFC 6238) allows the client to generate the 6-digit 2FA code locally, provided it possesses the shared secret. This secret is typically a Base32 string obtained from the Upstox security settings.[6]

By integrating a library such as pyotp or the specialized upstox-totp package, the trading system can mathematically derive the valid OTP for the current 30-second window. This eliminates the dependency on SMS or Email delivery, which are prone to latency and interception risks.

Mathematical Foundation of TOTP:

$$TOTP = \text{Truncate}(\text{HMAC-SHA1}(K, T))$$

Where K is the shared secret and T is the time counter ($T = \lfloor \frac{\text{Current Unix Time}}{30} \rfloor$).

This automated generation allows the LoginApi class in the Python SDK to seamlessly authenticate the session during the pre-market initialization phase.[8]

2.3 Token Lifecycle Management

The access_token returned by Upstox is valid for the entire trading day (typically until the next morning). A robust architecture must implement a token caching strategy.

1. **Storage:** Tokens should be serialized and stored securely (e.g., encrypted local file or

Redis cache). Hardcoding tokens is a critical security vulnerability.
2. **Validation:** Upon system startup, the bot should attempt a lightweight API call (e.g., get_profile or get_funds) using the cached token.[8]
3. **Refresh/Re-auth:** If the API returns a 401 Unauthorized error, the system must trigger the full login flow.

This "Check-then-Act" pattern prevents the algorithm from failing mid-trade due to an expired session, a scenario that could lead to significant financial loss if open positions cannot be managed.

---

3. Instrument Universe and Master Data Management

A quantitative trading system is only as effective as its map of the market. The Upstox API exposes the entire universe of tradable assets—spanning Equities, Futures, Options, Commodities, and Currencies—through massive master files. Efficiently managing this data is a "Big Data" engineering challenge within the micro-environment of a trading bot.

3.1 The Master Contract Database

Upstox provides the instrument master list as a GZIP-compressed JSON file, often exceeding hundreds of megabytes in uncompressed size.[10] This file contains the critical mapping between human-readable symbols (e.g., "NIFTY 25000 CE") and the machine-readable instrument_key (e.g., NSE_FO|12345) required for all API interactions.

3.1.1 Handling Large Datasets in Python

Loading the entire master list into a standard Python list of dictionaries is memory-inefficient and slow. The recommended approach utilizes the pandas library for high-performance data manipulation.[11]

Architectural Pattern for Master Data:

1. **Stream Download:** The file is downloaded as a binary stream to avoid loading the entire GZIP archive into RAM at once.
2. **In-Memory Decompression:** The gzip and io libraries facilitate on-the-fly decompression.
3. **Pandas Ingestion:** The data is loaded directly into a Pandas DataFrame.

```python
```

```python
import pandas as pd
import requests
import gzip
import io

def fetch_instrument_master_list():
    url = "https://assets.upstox.com/market-quote/instruments/exchange/complete.json.gz"
    response = requests.get(url, stream=True)
    if response.status_code == 200:
        with gzip.GzipFile(fileobj=io.BytesIO(response.content)) as f:
            df = pd.read_json(f)
        return df
    else:
        raise Exception(f"Failed to fetch master list: {response.status_code}")
```

Once loaded, this DataFrame acts as an in-memory database. Queries that would take milliseconds to execute via network calls (searching for a symbol) can be executed in microseconds using vectorized Pandas operations.[12]

3.2 Dynamic Symbol Resolution and ATM Logic

One of the most complex tasks in options trading is resolving the dynamic "At-The-Money" (ATM) strike price. Unlike stocks, which have a static symbol (e.g., "RELIANCE"), the ATM strike changes with every tick of the underlying asset.

The "Neuro-Quantum" and "Smart Indicator" strategies rely on selecting the correct strike to maximize gamma exposure while minimizing theta decay.[14] This requires a dynamic resolution algorithm.

3.2.1 The ATM Calculation Algorithm

To determine the ATM instrument key, the system must:

1. **Fetch Spot Price:** Retrieve the current market price of the underlying index (e.g., NIFTY 50) via the NSE_INDEX|Nifty 50 key.
2. **Round to Strike Interval:** NIFTY strikes are spaced by 50 points. The mathematical rounding logic must align with exchange specifications.

$$Strike_{ATM} = \text{Round}\left(\frac{Price_{Spot}}{50}\right) \times 50$$

3. **Filter Master Data:** Query the Master DataFrame for a contract that matches the strike_price, expiry (nearest Thursday), instrument_type (CE/PE), and segment (NSE_FO).[16]

Handling Edge Cases: The rounding logic provided in generic programming tutorials often fails in financial contexts due to floating-point errors. Explicit rounding logic using decimal or numpy is preferred to ensure that a spot price of 19,024.99 rounds down to 19,000, while 19,025.00 rounds up to 19,050 (or as per specific exchange rounding conventions).[17]

3.3 Option Chain Data Structure

While the master list provides static data, the Option Chain API provides the dynamic state of the market (Greeks, OI, PCR). The Upstox V3 Option Chain endpoint returns a nested JSON structure containing call_options and put_options objects for each strike.[19]

Navigating this structure requires precise parsing. The API response groups data by strike price. The Python application must flatten this hierarchy into a tabular format (Pandas DataFrame) to calculate metrics like the Put-Call Ratio (PCR) or Max Pain, which are essential inputs for the "Trap Detector" module in the War Room console.[21]

---

4. Historical Data Architecture and Quantitative Analysis

The backbone of the "Neuro-Quantum Brain" is historical data. Deep Learning models (LSTM) and Fractal analysis require continuous time-series data to identify regimes and patterns.

4.1 REST API Endpoints for Candle Data

Upstox distinguishes between "Historical" and "Intraday" data endpoints, a distinction that often confuses developers.

- **Historical Candle Data API:** Optimized for past data (previous days/months/years).
- **Intraday Candle Data API:** Optimized for the *current* trading day.[22]

This separation is an architectural decision to optimize database queries on the broker's side. The trading bot must seamlessly stitch these two data sources together to form a continuous array for analysis (e.g., calculating a 200-period Moving Average requires data from both today and previous days).

4.2 Handling Pagination and Rate Limits

A critical constraint identified in the API documentation is the limit on data retrieval. For minute-level granularity, the API typically restricts the response to a specific duration (e.g., 1 month or fewer candles) per request.[23]

To train the "Neuro-Quantum" LSTM model which requires significant historical context, the system must implement a "Back-fill" algorithm:

1. **Iterative Requesting:** The bot requests data in 30-day chunks, moving backward in time from the current date.
2. **Rate Limit Governance:** The Upstox API enforces strict rate limits (e.g., 3 requests per second). The fetcher loop must implement time.sleep() intervals or use an exponential backoff strategy to prevent IP bans.[24]
3. **Concatenation:** The resulting JSON payloads are concatenated into a single Pandas DataFrame, sorted by the timestamp index.

4.3 Data Normalization for Machine Learning

The raw API response is a list of lists: [[timestamp, open, high, low, close, volume, oi],...]. The "Neuro-Quantum" snippet [14] expects a Numpy array or Pandas Series. The conversion process involves:

1. **Type Casting:** Converting the timestamp string to a Python datetime object.
2. **Indexing:** Setting the timestamp as the DataFrame index to allow for time-based slicing (e.g., df.between_time('09:15', '15:30')).
3. **Cleaning:** Handling missing candles (dojis or gaps) which can distort Fractal Dimension calculations.

---

5. Advanced Quantitative Models: The Core Logic

The research materials provide three distinct "Brains" or logic engines. This section analyzes their theoretical underpinning and implementation within the Python ecosystem.

5.1 The Neuro-Quantum Brain (Hybrid LSTM-Fractal Model)

This model represents a fusion of stochastic calculus and deep learning. It posits that market regimes can be classified into "Trend" or "Chaos" using Fractal Geometry, and directional moves can be predicted using Neural Networks.[14]

5.1.1 Fractal Dimension (The Hurst Exponent)

The Hurst Exponent (H) is a measure of the long-term memory of a time series.

- $0.5 < H < 1.0$: Persistent behavior (Trending). A positive return is likely followed by a positive return.
- $0 < H < 0.5$: Anti-persistent behavior (Mean Reverting).
- $H = 0.5$: Brownian Motion (Random Walk).

The provided snippet [14] calculates H using the Rescaled Range (R/S) analysis or a simplified

standard deviation of lags. **Strategic Implication:** The bot uses this as a filter. If $H \approx 0.5$, the LSTM's prediction is discarded because the market is in a random walk phase, where patterns are statistically insignificant.

5.1.2 Shannon Entropy (Chaos Theory)

Shannon Entropy (S) measures the information content or uncertainty in the price distribution.

$$S = -\sum p_i \ln(p_i)$$

Where p_i is the probability of a specific return occurring. **Strategic Implication:** High entropy implies high disorder (Chaos). The "Neuro-Quantum" logic uses a threshold (e.g., $S > 0.8$) as a "Kill Switch" or "Trap Check".[14] If the market is too chaotic, the system halts trading to preserve capital, effectively filtering out "whipsaw" market conditions.

5.2 The Rocket Science Trend Kalman Filter

The Kalman Filter is a recursive algorithm that estimates the state of a dynamic system from a series of noisy measurements. In trading, the "state" is the true price trend, and the "noise" is the market microstructure volatility.[25]

Mechanism:

1. **Prediction Phase:** The filter predicts the next price based on the current estimated velocity.
2. **Correction Phase:** It compares the predicted price with the actual market price (measurement).
3. **Update Phase:** The difference (error) is used to update the estimates of position and velocity, weighted by a "Gain" factor.

Superiority over Moving Averages:

Unlike an Exponential Moving Average (EMA) which introduces lag, the Kalman Filter is designed to minimize lag while smoothing data. This allows the trading bot to enter trends earlier and exit faster when the "velocity" component flips, providing a significant edge in HFT environments.

5.3 The Smart Indicator Engine (CPR & Trap Zones)

This logic relies on the Central Pivot Range (CPR), a leading indicator derived from the

previous day's High, Low, and Close (OHLC).[15]

The Math:

- $\text{Pivot} = (High + Low + Close)/3$
- $BC = (High + Low)/2$
- $TC = (Pivot - BC) + Pivot$

The "Trap Zone" Concept:

The area between TC and BC is the CPR.

- **Narrow CPR:** Indicates low volatility the previous day, often leading to a breakout (Trend Day).
- **Wide CPR:** Indicates high volatility the previous day, often leading to a range-bound day (Sideways).

The algorithm explicitly checks if the current price is *inside* the CPR. If so, it flags a "No Trade Zone" (Trap). This rule-based filter prevents the system from getting chopped up in a non-directional market, acting as a secondary validation layer alongside the Entropy check.

---

6. Real-Time Market Data Ingestion: WebSocket V3

In high-frequency trading, the speed of data ingestion is the primary bottleneck. Upstox V3 addresses this with a WebSocket feed powered by Protocol Buffers.

6.1 Protobuf vs. JSON: An Architectural Comparison

Feature	JSON (V2/Legacy)	Protobuf (V3)
Format	Text-based (ASCII/Unicode)	Binary
Size	High (Field names repeated)	Extremely Low (Schema-based)
Parsing CPU Load	High (String parsing)	Low (Direct byte mapping)
Bandwidth	High	Low

Complexity	Low (Native support)	High (Requires compilation)

The shift to binary streams means the Python client receives a stream of bytes. It cannot simply json.loads() this data. It must use the generated MarketDataFeedV3_pb2.py class to deserialize the bytes into a Python object.[26]

6.2 The MarketDataStreamer Wrapper

To abstract this complexity, the upstox-python-sdk provides the MarketDataStreamer class.[28] This class manages the WebSocket lifecycle:

1. **Connection:** Establishes the WSS handshake.
2. **Authentication:** Automatically handles the redirection to the authorized WebSocket endpoint (a specific V3 requirement).[29]
3. **Subscription:** manages the instrument_keys subscription list (add/remove instruments dynamically).
4. **Callback Routing:** Routes the decoded data to user-defined on_message functions.

Handling Binary Data: The snippet [30] illustrates the critical step of using the .proto definition to parse the feed. Without the correct .proto file version (matching the API version), the decoding will fail or produce garbage data. This version coupling is a strict requirement of Protobuf architectures.

---

7. Order Execution Management System (OEMS)

Execution is where analysis transforms into financial reality. The Upstox V3 Order API offers precision control over trade execution.

7.1 Order Types and Parameters

The /v3/order/place endpoint accepts a complex JSON payload defining the trade parameters.[31]

Order Type	Logic	Use Case in Algorithm
MARKET	Execute immediately at any price.	"Panic Exit" or high-urgency entries (high slippage risk).
LIMIT	Execute at a specific price	Standard entries to control

	or better.	cost basis.
SL	Stop Loss Limit. Triggers at Price X, Limit at Y.	Protective stops to cap downside.
SL-M	Stop Loss Market. Triggers at Price X, Market Order.	Emergency stops (guaranteed exit, unknown price).

Contextual Nuance: The "Rocket Science" logic [25] generates a "Deviation Strength" signal. If the strength is high (high momentum), a MARKET order might be preferable to ensure fill. If the strength is low (mean reversion), a LIMIT order placed at the support level is optimal. The OEMS must dynamically select the order type based on the *quality* of the signal.

7.2 GTT (Good Till Triggered) Mechanics

For algorithmic systems, GTT orders provide server-side state management. Instead of the Python script constantly checking if price < stop_loss, the bot places a GTT order with Upstox.[32]

- **Benefit:** If the bot crashes, the internet fails, or the EC2 instance reboots, the Stop Loss remains active on the broker's server.
- **Mechanism:** GTT orders are "Single" or "OCO" (One Cancels Other). An OCO GTT allows placing both a Target and a Stop Loss simultaneously. If one hits, the other is cancelled automatically, providing a robust bracket order functionality.

7.3 Error Handling and UDAPI Codes

The API returns specific error codes (e.g., UDAPI100060 for Resource Not Found). A robust OrderManager class must implement a try-except block to catch ApiException.

- **Rate Limiting:** If the API returns 429 Too Many Requests, the system must pause.
- **Fund Check:** If 400 Bad Request indicates insufficient funds, the logic must halt trading to prevent a loop of rejected orders.[33]

---

8. The "War Room" Dashboard: Streamlit Architecture

The "War Room Console" described in the research material [21] represents the interface between the trader and the algorithm. Streamlit is the tool of choice for its ability to turn Python scripts into web apps, but its synchronous nature poses a challenge for real-time data.

8.1 The Producer-Consumer Concurrency Pattern

Streamlit scripts rerun from top to bottom on every interaction. A standard while True loop to fetch market data would block the UI, rendering buttons unresponsive.

To solve this, we employ a threaded Producer-Consumer architecture:

1. **Producer (Background Thread):** A dedicated Python thread runs the WebSocket client. It continuously receives ticks, decodes them, and updates a thread-safe shared object (e.g., st.session_state or a global dictionary).[34]
2. **Consumer (Main Thread/Streamlit):** The Streamlit script uses an auto-refresh mechanism (like streamlit_autorefresh) to reload the UI every second. On reload, it reads the latest values from the shared state and updates the metrics/charts.

Code Logic: The add_script_run_ctx function is essential here. Streamlit threads require a specific execution context to access st.session_state. Standard Python threads lack this. By attaching the context to the background thread, we allow the WebSocket worker to seamlessly write data that the UI thread can read.[36]

8.2 Cyberpunk UI/UX Design

The "War Room" aesthetic is achieved via custom CSS injection using st.markdown(unsafe_allow_html=True).

- **Visual Hierarchy:** Large, neon-colored metrics for P&L focus attention on critical financial health.
- **Threat Radar:** The "Entropy" value from the Neuro-Quantum brain is visualized as a gauge or radar chart. High entropy (Chaos) turns the radar red, providing an immediate visual cue to the trader that the "System is Locked" due to market conditions.[21]
- **Terminal Logs:** A scrolling text box simulates a hacker terminal, printing decision logs ("Kalman Filter: Trend Up", "Neuro-Quantum: Trap Detected"). This provides transparency into the "Black Box," allowing the trader to trust the bot's decisions.

9. Conclusion

The construction of a high-frequency algorithmic trading system on Upstox V3 is a multidisciplinary engineering feat. It requires the synthesis of:

- **Network Engineering:** Handling binary Protobuf streams and WebSocket stability.
- **Security Engineering:** Managing OAuth tokens and TOTP automation securely.
- **Quantitative Finance:** Implementing complex mathematical models like Hurst Exponents and Kalman Filters.
- **Software Architecture:** Designing non-blocking, multi-threaded applications (Streamlit) to visualize real-time data.

By adhering to the architectural patterns outlined in this report—specifically the separation of

data ingestion (Producer) from analysis and UI (Consumer), and the rigorous management of instrument master data—traders can build systems that are not only theoretically sound but operationally robust in the chaotic reality of live financial markets. The "Neuro-Quantum" and "Kalman" brains provide the edge, but the Upstox V3 infrastructure provides the essential speed and reliability required to monetize that edge.

Works cited

1. Market Data Feed - Developer API - Upstox, accessed on February 4, 2026, https://upstox.com/developer/api-documentation/get-market-data-feed/
2. Market Data Feed V3 - Developer API - Upstox, accessed on February 4, 2026, https://upstox.com/developer/api-documentation/v3/get-market-data-feed/
3. The Sniper Reversal (Ehlers Fisher Transform).pdf
4. Authentication | Upstox Developer API, accessed on February 4, 2026, https://upstox.com/developer/api-documentation/authentication/
5. How to authenticate using Authentication API - Developer API - Upstox Community, accessed on February 4, 2026, https://community.upstox.com/t/how-to-authenticate-using-authentication-api/3019
6. Upstox TOTP Python SDK — Upstox TOTP 1.0.7 documentation, accessed on February 4, 2026, https://upstox-totp.readthedocs.io/
7. Any method to get the access code with Python instead of doing it manually with the URL?, accessed on February 4, 2026, https://community.upstox.com/t/any-method-to-get-the-access-code-with-python-instead-of-doing-it-manually-with-the-url/6359
8. Official Python SDK for accessing Upstox API - GitHub, accessed on February 4, 2026, https://github.com/upstox/upstox-python
9. upstox-python-sdk 2.3.0 - PyPI, accessed on February 4, 2026, https://pypi.org/project/upstox-python-sdk/2.3.0/
10. Instruments | Upstox Developer API, accessed on February 4, 2026, https://upstox.com/developer/api-documentation/instruments/
11. What is the correct endpoint in V2/V3 to fetch the full list of instruments dynamically via API (preferably segment-wise for NSE, MCX, BSE)? - Upstox Community, accessed on February 4, 2026, https://community.upstox.com/t/what-is-the-correct-endpoint-in-v2-v3-to-fetch-the-full-list-of-instruments-dynamically-via-api-preferably-segment-wise-for-nse-mcx-bse/11780
12. How to read a large json in pandas? - Stack Overflow, accessed on February 4, 2026, https://stackoverflow.com/questions/46790390/how-to-read-a-large-json-in-pandas
13. What is the fastest way to filter over 2.5gb Json file? - Stack Overflow, accessed on February 4, 2026, https://stackoverflow.com/questions/67045049/what-is-the-fastest-way-to-filter-over-2-5gb-json-file

14. THE NEURO-QUANTUM BRAIN (The Secret Weapon).pdf
15. The Smart Indicator Engine (Precision Core).pdf
16. How to Find the Instrument Key Associated with Stock, Option, Index and Commodity, accessed on February 4, 2026, https://community.upstox.com/t/how-to-find-the-instrument-key-associated-with-stock-option-index-and-commodity/7285
17. pandas.DataFrame.round — pandas 3.0.0 documentation - PyData |, accessed on February 4, 2026, https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.round.html
18. [pandas] How to round up to the nearest 0.5? : r/learnpython - Reddit, accessed on February 4, 2026, https://www.reddit.com/r/learnpython/comments/rwj1ir/pandas_how_to_round_up_to_the_nearest_05/
19. Option Contracts - Developer API - Upstox, accessed on February 4, 2026, https://upstox.com/developer/api-documentation/get-option-contracts/
20. Put/Call Option Chain - Developer API - Upstox, accessed on February 4, 2026, https://upstox.com/developer/api-documentation/get-pc-option-chain/
21. WAR ROOM CONSOLE.pdf
22. Historical Data - Developer API - Upstox, accessed on February 4, 2026, https://upstox.com/developer/api-documentation/historical-data/
23. Historical Candle Data V3 - Developer API - Upstox, accessed on February 4, 2026, https://upstox.com/developer/api-documentation/v3/get-historical-candle-data/
24. Historical-candle-data - Developer API - Upstox Community, accessed on February 4, 2026, https://community.upstox.com/t/historical-candle-data/7769
25. The Rocket Science Trend Kalman Filter.pdf
26. Help me with Websocket Feed v3 - General - Upstox Community, accessed on February 4, 2026, https://community.upstox.com/t/help-me-with-websocket-feed-v3/9933
27. I want to dynamically update the subscription to websocket. Review the code give your solution - Developer API - Upstox Community, accessed on February 4, 2026, https://community.upstox.com/t/i-want-to-dynamically-update-the-subscription-to-websocket-review-the-code-give-your-solution/3873
28. MarketDataStreamerV3 error - Developer API - Upstox Community, accessed on February 4, 2026, https://community.upstox.com/t/marketdatastreamerv3-error/12218
29. Problem with the websocket - Developer API - Upstox Community, accessed on February 4, 2026, https://community.upstox.com/t/problem-with-the-websocket/10329
30. The market data feed response requires decoding using protobuf, utilizing the provided proto file. You'll need to translate the Market Data Proto File into the appropriate format compatible with your specific programming language - Upstox Community, accessed on February 4, 2026, https://community.upstox.com/t/the-market-data-feed-response-requires-deco

ding-using-protobuf-utilizing-the-provided-proto-file-youll-need-to-translate-the-market-data-proto-file-into-the-appropriate-format-compatible-with-your-specific-programming-language/9448
31. Place Order V3 | Upstox Developer API, accessed on February 4, 2026, https://upstox.com/developer/api-documentation/v3/place-order/
32. Place GTT Order - Developer API - Upstox, accessed on February 4, 2026, https://upstox.com/developer/api-documentation/place-gtt-order/
33. How to get option chain for Nifty using python api - Upstox Community, accessed on February 4, 2026, https://community.upstox.com/t/how-to-get-option-chain-for-nifty-using-python-api/1290
34. Create a background scheduled task which will update the cache and auto update UI, accessed on February 4, 2026, https://discuss.streamlit.io/t/create-a-background-scheduled-task-which-will-update-the-cache-and-auto-update-ui/80487
35. Live plot from a thread - Using Streamlit, accessed on February 4, 2026, https://discuss.streamlit.io/t/live-plot-from-a-thread/247
36. Multithreading for StreamLit and Kite.connect() - Stack Overflow, accessed on February 4, 2026, https://stackoverflow.com/questions/76472211/multithreading-for-streamlit-and-kite-connect